Lesson 4: Direct Hit

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: detecting when things touch. Your job is to add one enemy to the screen and make your bullet destroy it on contact - the first piece of your capstone's actual combat loop.

Open your own [YourName]Game project from Lesson 3 - don't start a new one. Everything this week builds on the player and bullet you already have.

Tip: this lesson's Assets folder has a ready-to-use placeholder enemy image (enemy_invader_red.png) and a new sound effect (sfx_hit.wav) - you don't need to find or make your own art today.

Setup

1. Open your [YourName]Game project from Lesson 3.
2. Import enemy_invader_red.png (this lesson's Assets folder) into the Content Browser, then build M_Enemy_Red: a new Material, Shading Model = Unlit, a Texture Sample using enemy_invader_red.png. This one needs a second node too - right-click, search "VectorParameter," add one, rename it Color, and leave its default value White. Multiply the Texture Sample's RGB output by the Color parameter (right-click > Multiply), then wire that result into Emissive Color. With Color left at White, multiplying does nothing and the enemy shows the plain red sprite - but it gives Lesson 5's striped-row idea and this lesson's own Visual idea a real, working tint to grab onto later, the same way a Sprite2D's modulate or a SpriteRenderer's color tints an actual sprite in the other engines.
3. Create a new Blueprint class (Actor parent), name it BP_Enemy. Add a Static Mesh Component to it and set its shape to a small basic Shape, distinct from the player's Cube and BP_Bullet's Sphere from Lesson 3. Assign M_Enemy_Red to it in the Material slot - a default Shape's built-in material is Lit and renders solid black with this project's lights removed.
4. Set BP_Enemy's collision component's Collision Presets to OverlapAllDynamic (Details panel, Collision section). Then open BP_Bullet from Lesson 3 and set its collision component to the same OverlapAllDynamic preset - both objects need this, not just the one whose overlap event you're about to build. The default preset a fresh Shape starts with (BlockAllDynamic) makes two dynamic objects physically block each other instead of generating an overlap event, so without this change on both sides, Event ActorBeginOverlap below will never fire no matter how correctly it's wired - and OverlapAllDynamic sets Generate Overlap Events on automatically as part of the preset, so there's no separate checkbox to hunt down.

5. While you're in BP_Bullet, find the Add Actor World Offset node from Lesson 3's movement and check its Sweep box (click the small arrow on the node to reveal it if it's not already visible). Without Sweep, that node relocates BP_Bullet each frame without ever testing for collision along the way, so it can pass straight through BP_Enemy without the overlap ever registering - Sweep is what makes the move actually check for what it's passing into.
6. Drag one instance of BP_Enemy into the level near the top of the play area.

Level 1 - Minimal (MVP)

A single enemy sits on screen. When your bullet touches it, both the bullet and the enemy disappear. This is a complete finish for today - reaching this is enough to move on to next week.

7. In BP_Enemy's Event Graph, add an Event ActorBeginOverlap node, feeding a "Cast To BP_Bullet" node on the Other Actor pin - on a successful cast, wire a "Destroy Actor" node targeting the cast result (the bullet), and a second "Destroy Actor" node targeting Self.
8. Click Compile, then Play and shoot the enemy - both the bullet and the enemy should disappear on contact.

```
BP_Enemy's Event Graph:
Event ActorBeginOverlap(OtherActor) ->
  Cast To BP_Bullet(OtherActor) ->
    Success: Destroy Actor(target = Cast result)
             Destroy Actor(target = Self)
```

Alternate Path: C++ (code, optional advanced track)

The same overlap-detection pattern works in C++ instead of Blueprint nodes.

9. In a new ACppEnemy class (Tools > New C++ Class, Actor parent, same as CppBullet in Lesson 3), bind a function to the OnActorBeginOverlap delegate, typically in BeginPlay().
10. In that bound function, cast OtherActor to ACppBullet - on a successful cast, call Destroy() on both the cast result and this actor.
11. Compile, drag a test setup into the level in place of your existing Blueprint enemy, Play, and test the same way as the Blueprint path.

```
// ACppEnemy.h
virtual void BeginPlay() override;
UFUNCTION()
void OnOverlap(AActor* OverlappedActor, AActor* OtherActor);

// ACppEnemy.cpp
void ACppEnemy::BeginPlay() {
    Super::BeginPlay();
    OnActorBeginOverlap.AddDynamic(this, &ACppEnemy::OnOverlap);
}

void ACppEnemy::OnOverlap(AActor* OverlappedActor, AActor* OtherActor) {
    if (Cast<ACppBullet>(OtherActor)) {
        OtherActor->Destroy();
        Destroy();
    }
}
```

This mirrors the Blueprint path's Event ActorBeginOverlap and Cast To BP_Bullet nodes - same underlying engine calls, just from code. Level 2/3: Destroy() is safe to call more than once in C++ too, and UGameplayStatics::PlaySound2D() plus a dynamic material instance cover the Level 3 sound and visual ideas.

Level 2 - Complete (Alpha/Beta)

The enemy is destroyed cleanly and exactly once, even if you fire several bullets in a row - no errors, no lingering references, and a bullet that misses still behaves exactly like it did last week.

- Fire several bullets in a row, including some that miss - each hit destroys exactly one BP_Bullet and BP_Enemy, and any bullet that misses keeps flying and destroys itself off-screen exactly like it did in Lesson 3.
- Destroy Actor is safe to call more than once on the same Actor - a repeat call on an Actor that's already pending destruction is simply ignored - so even the rare case of two bullets overlapping the enemy on the same tick doesn't cause an error.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: add a Play Sound 2D node (using an imported sfx_hit.wav Sound Wave asset) right after the successful Cast, before either Destroy Actor node.
- Visual: in BP_Enemy's own BeginPlay, create a dynamic material instance and set its Color parameter (from M_Enemy_Red in Setup) to a darker red, giving the enemy its own distinct tint on top of its already-distinct sprite - the same modulate/tint idea the other engines use, just reached through Unreal's Material system instead of a Sprite property.
- Structure: keep everything about how the enemy responds to being hit - the Cast, both Destroy Actor nodes, the sound - inside BP_Enemy's own Event Graph. BP_Bullet doesn't need any new nodes at all this week; it only needed to exist as something BP_Enemy's overlap event could detect.

If Something's Wrong

- Nothing happens on contact, no errors anywhere: this is almost always Collision Presets, not the overlap event's wiring - confirm BOTH BP_Enemy's and BP_Bullet's collision components are set to OverlapAllDynamic (Details panel, Collision section), not left on the default BlockAllDynamic. Two Actors visibly touching while either one is still set to Block instead of Overlap will never fire Event ActorBeginOverlap, and produces no error of any kind.
- Collision Presets are correct on both, but it still doesn't fire: check that Sweep is enabled on BP_Bullet's Add Actor World Offset node from Lesson 3 - without it, that node teleports BP_Bullet each frame without testing for collision along the way, so a fast-moving bullet can pass straight through BP_Enemy without ever registering the overlap.
- Everything above checks out and it still doesn't fire: select BP_Enemy's and BP_Bullet's Static Mesh components in their own Blueprint editors (not an instance in the level) and confirm each actually has a mesh assigned in its Static Mesh field - a component left on None has no real geometry to collide with, even if every collision setting on it is otherwise correct.
- Cast To BP_Bullet always fails: confirm the enemy is actually overlapping the bullet's collision shape, not just its visible mesh.

- Enemy renders solid black instead of red: open M_Enemy_Red and confirm Texture Sample -> Multiply -> Emissive Color is wired all the way through, that enemy_invader_red.png is actually assigned as the Texture Sample's Texture, and that the Color VectorParameter's default is White, not black (multiplying by black would zero out the sprite entirely).
- This is a heavy engine on our list - if a laptop struggles, it's fine to observe this week's demo and build the exercise in a lighter engine instead.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal